Jornada 2: instanciación de memorias y procesador PicoRV32

Daniel Rosso - Pablo Madoery

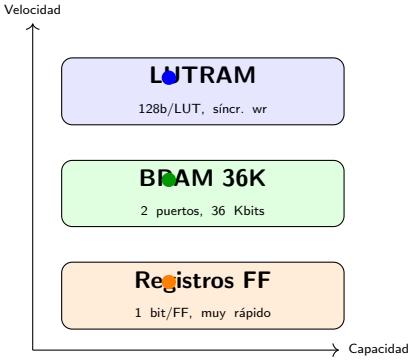
Fundación Fulgor

Miércoles 26 de agosto de 2026

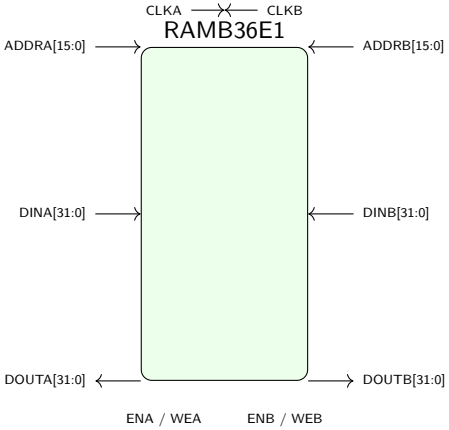
Recursos de memoria en un Artix-7

Tres formas de implementar memoria:

1. **Distributed RAM** (LUTRAM)
   - Usando LUTs como celdas de memoria
   - Lectura combinacional, escritura síncrona
   - Limitada: 400 Kbits en Artix-7 35T
   - *Uso típico: banco de registros, FIFOs chicas — **no** caches*
2. **Block RAM** (BRAM)
   - Bloques dedicados
   - Dos puertos independientes (port A / port B)
   - Inferencia automática o instancia manual
   - *Uso típico: caches, boot ROM, buffers medianos-grandes*
3. **UltraRAM** (solo en UltraScale+)
   - SRAM: 288 Kb c/u (BRAM: 36 Kb)
   - Cascadeable sin gastar LUTs extra
   - *Uso típico: buffers muy grandes (framebuffers)*



Arquitectura interna de la BRAM 36K



Modos de configuración:

- **Simple Dual-Port** (SDP): un puerto escribe, otro lee
- **True Dual-Port** (TDP): ambos leen y escriben
- **ROM**: sin escritura, inicializado con `$readmemh`

Parámetros clave:

- `READ_WIDTH_A/B`: ancho del bus de lectura
- `WRITE_WIDTH_A/B`: ancho del bus de escritura
- `INIT_xx`: contenido inicial (hasta 256 líneas de 256 bits)
- `DO_REG = 1`: agregar registro de salida (+1 ciclo, mejora timing)

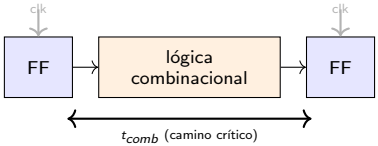
Timing: por qué DO_REG mejora el máximo reloj

El período de reloj tiene un piso:

$$T_{clk} \geq t_{cq} + t_{comb} + t_{setup}$$

- t_{cq}: lo que tarda un flip-flop en propagar el dato tras el flanco
- t_{comb}: el camino combinacional más largo entre dos flip-flops — el **camino crítico**
- t_{setup}: cuánto antes del flanco tiene que estar estable el dato en el flip-flop destino

Si t_{comb} es muy grande, no hay T_{clk} que alcance $\Rightarrow$ violación de setup: el flip-flop puede capturar un dato metaestable.



Por eso DO_REG = 1

Sin registro de salida, leer la BRAM es un camino combinacional largo (decodificador de dirección + array) que se suma al resto del datapath. Agregar un FF a la salida parte ese camino en dos: **+1 ciclo de latencia**, pero t_{comb} mucho más chico $\Rightarrow$ mayor F_{max}.

Inferencia de BRAM: el estilo que entiende Vivado

```
module bram_sp #(
  parameter DATA_W = 32,
  parameter ADDR_W = 10,
  parameter INIT_FILE = ""
)(
  input  wire                clk,
  input  wire                ena,
  input  wire                wea,
  input  wire [ADDR_W-1:0] addr,
  input  wire [DATA_W-1:0] din,
  output reg  [DATA_W-1:0] dout
);
  reg [DATA_W-1:0] mem [0:(1<<ADDR_W)-1];

  initial if (INIT_FILE != "")
    $readmemh(INIT_FILE, mem);

  always @(posedge clk) begin
    if (ena) begin
      if (wea) mem[addr] <= din;
      dout <= mem[addr];
    end
  end
endmodule
```

Por qué Vivado lo infiere como BRAM:

- `reg` indexado con array → memoria, no flip-flops sueltos
- `dout` es un `reg` actualizado en `posedge clk` → salida **registrada**
- `ADDR_W = 10` → $2^{10} = 1024$ palabras = 4 KB

`INIT_FILE`:

- Parámetro opcional: `""` = sin inicializar
- Si viene seteado, `$readmemh` carga el contenido en elaboración

Latencia

`dout` tarda **1 ciclo** en reflejar la lectura — clave para el `mem_ready` del lab.

BRAM como ROM de instrucciones (boot ROM)

```
module boot_rom (
  input  wire        clk,
  input  wire        ena,
  input  wire [9:0]  addr,
  output reg  [31:0] dout
);
  reg [31:0] mem [0:1023];

  initial begin
    mem[0] = 32'h00001137;
    mem[1] = 32'hffc10113;
    mem[2] = 32'h008000ef;
  end

  always @(posedge clk)
    if (ena) dout <= mem[addr];
endmodule
```

Una alternativa a `bram_sp`:

- Mismo estilo de inferencia, pero **sin puerto de escritura**: solo lectura en runtime
- El contenido se fija con `initial` en vez de `$readmemh` → un valor por instrucción, generado por script
- En este lab usamos `bram_sp` con `INIT_FILE` en su lugar — más simple para regenerar el firmware

Flujo de generación

`boot.bin` → `gen_boot_rom.py` → `boot_rom.v` → Vivado lo infiere como BRAM inicializada

Inicializar BRAM desde un archivo HEX

```
// $readmemh: carga enteros
initial $readmemh("boot.hex", mem);

// $readmemb: igual pero en binario
// $readmemh("file.hex", mem, start, end): rango opcional
```

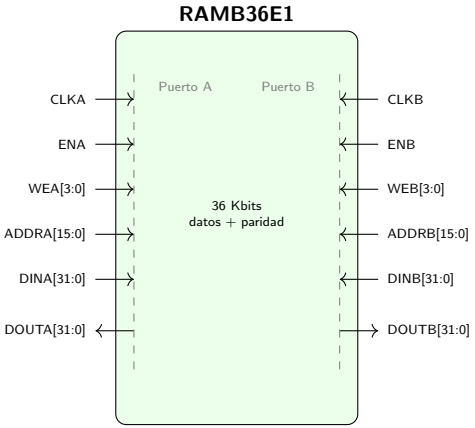
Formato de `boot.hex`**:**

```
00001137
ffc10113
008000ef
00000013
...
```

Generado por `bin2hex.py`**:**

```
import sys, struct
data = open(sys.argv[1], 'rb').read()
# Padding a múltiplo de 4 bytes
while len(data) % 4: data += b'\x00'
for i in range(0, len(data), 4):
    word, = struct.unpack_from('<I', data, i)
    print(f'{word:08x}')
```

Puertos A y B: señales y dominios de reloj independientes



Características clave:

- Es una **memoria de doble puerto**: dos accesos completamente independientes (A y B) al mismo contenido, cada uno con su propio address/datos/control
- **CLKA** y **CLKB** completamente independientes
  $\Rightarrow$ BRAM es un *verdadero* cruce de dominio de reloj (*asynchronous dual-port*)
- **WE** es bus de 4 bits: un bit por byte (byte-enable)
- **EN** habilita el puerto; sin EN activo, la salida se mantiene
- **RST** (no mostrado): reset síncrono de la salida DOUT (no del contenido)
- **REGCE**: habilita el registro de pipeline de salida cuando `DO_REG=1`

Por qué usar BRAM como ROM: fetch de instrucciones RISC-V

Requisitos del procesador:

- PicoRV32 necesita una instrucción por ciclo (32 bits)
- El bus de instrucciones es independiente del bus de datos (Harvard interno)
- La dirección de reset es 0x00000000

Por qué BRAM y no LUTRAM:

- 4 KB de código = 1024 palabras de 32 bits = 1 BRAM (eficiente)
- LUTRAM requeriría ∼512 LUT-6 para el mismo tamaño

Costo comparado (4 KB de código):

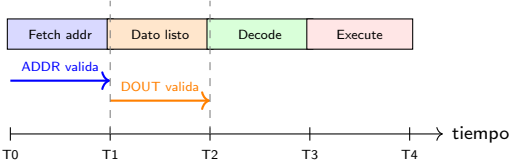
- **BRAM:** 1 × RAMB36E1 — < 1 % de las 50 disponibles en el Artix-7 35T
- **LUTRAM:** ∼512 × LUT-6 — ∼2.5 % de los LUTs del chip

Inicialización con `$readmemh`

El contenido de la BRAM se embebe en el bitstream de Vivado. Al programar la FPGA, el firmware ya está en la memoria — no se necesita flash externa.

Latencia de BRAM y su impacto en el PicoRV32

Latencia de la BRAM = 1 ciclo de reloj:



- **T0–T1**: el CPU presenta `ADDR` (fase *Fetch addr*)
- **T1–T2**: la BRAM tarda 1 ciclo en resolver la dirección ⇒ `DOUT` recién es válido en T2 (*Dato listo*)
- **T2–T3**: con el dato ya válido, el CPU decodifica la instrucción
- **T3–T4**: ejecuta con el opcode decodificado

Interfaz valid/ready de PicoRV32:

- El core presenta `mem_valid`, espera `mem_ready`
- La BRAM tarda **1 ciclo** ⇒ 1 wait state
- CPI aumenta en 1 por cada fetch/load/store

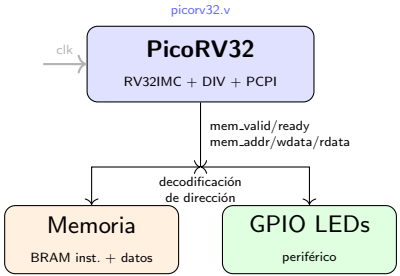
Cuidado en el lab

Si el CPU no espera `mem_ready`, lee `dout` un ciclo antes de que sea válido.

PicoRV32: procesador RISC-V de tamaño mínimo

Características clave:

- ISA: RV32I (enteros 32 bits) + extensiones opcionales
- **Un solo archivo Verilog** (`picorv32.v`, $\sim$ 3000 líneas)
- Interfaz de memoria nativa o AXI-4 Lite
- Parámetros de configuración: MUL, DIV, PCPI, IRQ, etc.
- Ciclos por instrucción: 3–5 CPI (sin pipeline, predecible)
- En Artix-7: $\sim$250 LUTs, hasta $\sim$130 MHz



En este curso instanciamos 2 cores:

- Core 0: generador PRBS + manager (UART)
- Core 1: verificador PRBS + cálculo de BER

PicoRV32: microarquitectura — FSM sin pipeline

No hay pipeline convencional:

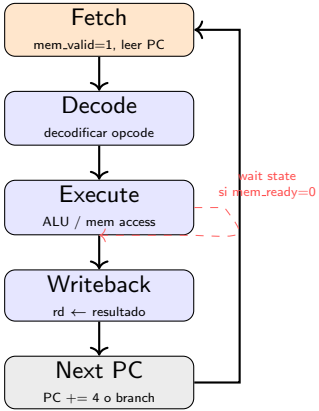
- PicoRV32 es una FSM de ∼40 estados
- Ejecuta una instrucción a la vez
- **CPI** (*Ciclos Por Instrucción*) promedio: **3–5 ciclos**
- Sin data hazards: el diseño es inherentemente seguro

Ventajas:

- Área mínima: ∼2000 LUTs en Artix-7
- Fácil de verificar formalmente

Desventajas:

- Throughput bajo vs. cores con pipeline
- La multiplicación consume 34 ciclos (shift-add en software, sin usar DSP48)



PicoRV32: CPI por instrucción

CPI: ciclos de reloj promedio que tarda el core en completar una instrucción.

Ciclos por instrucción (CPI)

Instrucción	CPI
ALU (ADD, AND, ...)	3
LOAD (lw, lh, lb)	5
STORE (sw, sh, sb)	4
Branch (no taken)	4
Branch (taken)	4
JAL / JALR	3
LUI / AUIPC	2
MUL (ENABLE_MUL=1)	34
DIV (ENABLE_DIV=1)	66

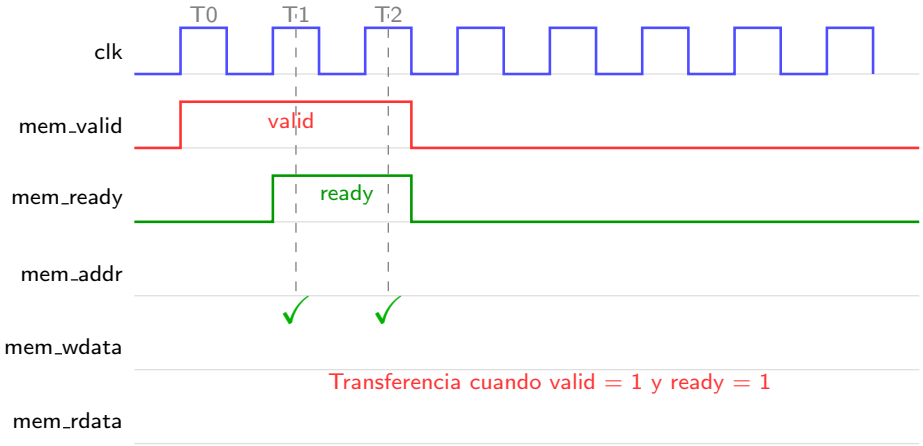
Nota: cada wait state en memoria suma 1 ciclo a LOAD/STORE.

Impacto en el diseño

En nuestro SoC a 100 MHz:

- Ciclo = 10 ns
- ADD típico = 30 ns efectivo
- DIV de 32 bits = 660 ns
- STORE = 4 ciclos = 40 ns
  $\rightarrow$ LEDs pueden actualizarse a $\sim$25 MHz (limitado por firmware)

Interfaz de memoria: valid/ready (timing)



Idea clave: la transferencia ocurre en los ciclos donde ambas señales coinciden en alto.

Interfaz de memoria: valid/ready (uso en PicoRV32)

```
// Señales principales
output reg          mem_valid;
input  wire         mem_ready;
output reg  [31:0] mem_addr;
output reg  [31:0] mem_wdata;
output reg  [ 3:0] mem_wstrb;
input  wire [31:0] mem_rdata;
// Lectura
mem_wstrb = 0;
// Escritura
mem_wstrb = 4'b1111; // word
// Handshake
if (mem_valid && mem_ready) begin
    // transferencia completada
end
// Stall automático
// mem_ready = 0 -> el core espera
```

mem_wstrb: byte-enable y mapeo a instrucciones

mem_wstrb[3:0] — codificación

wstrb	Operación	Instrucción C
4'b0000	Lectura	lw/lh/lb
4'b1111	Write word	sw
4'b0011	Write halfword bajo	sh (addr par)
4'b1100	Write halfword alto	sh (addr+2)
4'b0001	Write byte 0	sb addr[1:0]=00
4'b0010	Write byte 1	sb addr[1:0]=01
4'b0100	Write byte 2	sb addr[1:0]=10
4'b1000	Write byte 3	sb addr[1:0]=11

Arquitectura little-endian:

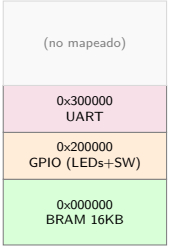
- mem_addr es **siempre alineada a word** (bits [1:0] = 0)
- El desplazamiento sub-word se codifica en wstrb
- Para lecturas parciales (lb/lh): el periférico devuelve la word completa y PicoRV32 extrae el campo internamente

Implicación para periféricos

Un periférico de registro de 32 bits puede ignorar wstrb si solo acepta accesos tipo sw (word completa). Pero una BRAM real debe usar wstrb como write enable por byte.

Mapa de memoria del SoC

Dirección	Periférico	Acceso
0x0000_0000 -- 0x0000_3FFF	BRAM (16 KB) instr. + datos	R/W
0x0020_0000 0x0020_0004	GPIO_LEDS GPIO_SW	W R
0x0030_0000 0x0030_0004	UART DATA UART STATUS	R/W R



Este es el mapa que va a decodificar el TODO 1

Cada región es lo que compara `sel_bram`/`sel_gpio`/`sel_uart` contra `mem_addr` — volvé a esta tabla mientras completás el lab.

TODO 1 — Decodificador de dirección

```
// mem_valid evita activar
// hardware con el bus inactivo
wire sel_bram = mem_valid &&
  (mem_addr[31:14] == 18'h0);
  // 0x000000-0x003FFF

wire sel_gpio = mem_valid &&
  (mem_addr[31:8] == 24'h002000);
  // 0x00200000

wire sel_uart = mem_valid &&
  (mem_addr[31:20] == 12'h003);
  // 0x300000-0x3FFFFF
```

Notas de implementación:

- Una señal `sel_*` por periférico, todas combinacionales
- Compara los bits altos de `mem_addr` contra el rango de cada uno (ver mapa de memoria)
- Mismo patrón para los tres — copiá y ajustá el rango

Sin esto, nada responde a las direcciones que pide el CPU: el primer paso de la integración.

TODO 2 — El módulo y el CPU

```
module soc_minimal (
  input  wire clk, resetn,
  output wire [3:0] led
);
  // Senales del bus
  wire [31:0] mem_addr, mem_wdata,
              mem_rdata;
  wire [3:0]  mem_wstrb;
  wire        mem_valid, mem_ready;

  picorv32 #(
    .ENABLE_MUL(1), .ENABLE_DIV(1),
    .PROGADDR_RESET(32'h0),
    .STACKADDR(4096*4)
  ) cpu (
    .clk(clk), .resetn(resetn),
    .mem_valid(mem_valid),
    .mem_ready(mem_ready),
    .mem_addr(mem_addr),
    .mem_wdata(mem_wdata),
    .mem_rdata(mem_rdata),
    .mem_wstrb(mem_wstrb)
  );
```

Notas de implementación:

- Declarar las 6 señales del bus (`mem_*`)
- Instanciar `picorv32` y conectarlas por nombre
- `STACKADDR = 4096*4`: el SP arranca al final de la BRAM

Todavía no compila solo

El CPU pide memoria (`mem_valid`) pero nadie le contesta todavía — faltan las instancias (próximas slides).

TODO 3 — Instanciar la BRAM

```
// BRAM 16 KB, firmware cargado
// con $readmemh en bram_sp.v
wire [31:0] bram_rdata;
bram_sp #(.ADDR_W(12),
    .INIT_FILE("fw/boot.hex")
) u_bram (
  .clk(clk), .ena(sel_bram),
  .wea(|mem_wstrb),
  .addr(mem_addr[13:2]),
  .din(mem_wdata),
  .dout(bram_rdata));
```

Notas de implementación:

- `bram_sp` es el mismo módulo visto en la sección de BRAM — ahora lo instanciás vos
- `ena = sel_bram`: solo activa cuando el decodificador lo selecciona
- `addr` se desplaza 2 bits: byte-address → word-address

TODO 4 — GPIO: LEDs y switches

```
wire gpio_is_sw = mem_addr[2];
  // 0x200000=LEDs(W)
  // 0x200004=SW(R)

reg [3:0] gpio_reg;
always @(posedge clk)
  if (!resetn)
    gpio_reg <= 4'b0;
  else if (sel_gpio && !gpio_is_sw
           && |mem_wstrb)
    gpio_reg <= mem_wdata[3:0];
assign led = gpio_reg;
```

Notas de implementación:

- `gpio_is_sw` distingue el offset LEDs (0x0) de switches (0x4) dentro de la misma región
- El registro se actualiza solo escribiendo al offset de LEDs
- La **lectura** de `sw` se resuelve en el mux del último TODO, no acá

TODO 5 — Instanciar la UART

```
wire [31:0] uart_rdata;
wire        uart_ready;
uart_regs #(
  .CLK_HZ(CLK_HZ), .BAUD(BAUD)
) u_uart (
  .clk(clk), .resetn(resetn),
  .sel(sel_uart),
  .reg_sel(mem_addr[2]),
  .wstrb(mem_wstrb),
  .wdata(mem_wdata),
  .rdata(uart_rdata),
  .ready(uart_ready),
  .rx(rx), .tx(tx));
```

Notas de implementación:

- `uart_regs` es el mismo periférico visto en la sección de UART
- `reg_sel = mem_addr[2]` distingue DATA de STATUS, igual patrón que `gpio_is_sw`
- `ready` ya viene resuelto del módulo — se usa tal cual en el mux

TODO 6 — mem_rdata y mem_ready: cerrando el bus

```
assign mem_rdata =
  sel_bram ? bram_rdata :
  sel_uart ? uart_rdata :
  sel_gpio ? (gpio_is_sw
    ? {28'b0, sw}
    : {28'b0, gpio_reg}) :
  32'h0;

// BRAM: salida registrada =
// 1 ciclo de latencia
reg bram_ready_r;
always @(posedge clk)
  bram_ready_r <=
    sel_bram && !mem_ready;

assign mem_ready = bram_ready_r
  || sel_gpio || uart_ready;
```

Notas de implementación — cierra la integración:

- `mem_rdata`: mux combinacional, uno por periférico
- `mem_ready`: GPIO/UART responden inmediato, BRAM necesita 1 ciclo de retardo

El bug clásico

`mem_ready = sel_bram || ...` (sin registrar) haría que el CPU lea `bram_rdata` un ciclo **antes** de que sea válido.

Top level de síntesis: soc_top.v y el pinout de la Arty A7

soc_top.v (dado)

```
module soc_top (
  input  wire        clk,    // 100 MHz
  input  wire        btn0,   // reset, activo alto
  output wire [3:0] led,
  input  wire [3:0] sw,
  input  wire        rx,
  output wire        tx
);
  wire resetn = ~btn0;  // PicoRV32: reset activo bajo

  soc_minimal u_soc (
    .clk(clk), .resetn(resetn),
    .led(led), .sw(sw),
    .rx(rx),   .tx(tx)
  );
endmodule
```

soc_top.v envuelve a soc_minimal (el módulo que estás completando) y le da nombres de puerto que coinciden con las restricciones físicas — es lo que Vivado sintetiza.

Pinout real (constraints/arty_a7.xdc):

Puerto	Pin(es)	Función
clk	E3	reloj 100 MHz
btn0	D9	BTN0, reset
led[3:0]	H5,J5,T9,T10	LD0..LD3
sw[3:0]	A8,C11,C10,A10	SW0..SW3
rx	G13 (JA1)	UART RX (desde FTDI)
tx	B11 (JA2)	UART TX (hacia FTDI)

TODO 7 — completar constraints/arty_a7.xdc

soc_top.v viene dado; el .xdc trae la estructura armada pero con los PACKAGE_PIN en blanco (???) — completalos con esta misma tabla. Es lo que le dice a Vivado **qué pin físico** es cada señal.

Laboratorio: SoC PicoRV32 — BRAM + GPIO + switches + UART

Objetivo

Integrar el procesador con la BRAM, el GPIO (LEDs + switches) y la UART en un SoC completo. El firmware — un generador PRBS con inyección de error controlada por switches — viene ya compilado (`fw/boot.hex`): escribirlo es el contenido de la Jornada 3.

Ya dado (no tocar):

- `picorv32.v`, `bram_sp.v`, `uart_regs.v` (+`uart_tx/rx.v`) — el interior de cada módulo
- `rtl/soc_top.v` (wrapper de síntesis)
- `fw/boot.hex` / `boot_sim.hex` (firmware ya compilado)

A completar en `soc_minimal.v`:

1. Decodificador (`sel_bram`/`sel_gpio`/`sel_uart`)
2. Instanciar `picorv32`
3. Instanciar `bram_sp`
4. GPIO: registro de LEDs + lectura de switches
5. Instanciar `uart_regs`
6. Mux `mem_rdata` y `mem_ready`

Flujo de trabajo:

1. TODOs 1–6 → `make sim` (no necesita constraints): verificar escrituras al GPIO y banner por UART
2. TODO 7 → `make synth` → sintetizar (Vivado batch)
3. `make gui` → Vivado GUI (también permite correr la simulación)
4. `make program` → programar Arty A7 con `fw/boot.hex`
5. Mover SW[3:0] y ver que el nivel reportado por UART cambia en vivo — confirma que tu hardware (decodificador + switches + pinout) funciona

Testbench del SoC mínimo (tb_soc_minimal.v)

```
`timescale 1ns/1ps
module tb_soc_minimal;
    reg clk = 0, resetn = 0;
    wire [3:0] led;
    soc_minimal dut (.clk(clk), .resetn(resetn), .led(led), .sw(4'd8));
    always #5 clk = ~clk;  // 100 MHz

    // Detectar escrituras al GPIO (direccion 0x200000)
    always @(posedge clk)
        if (dut.mem_valid && dut.mem_ready && |dut.mem_wstrb
            && dut.mem_addr[31:8] == 24'h002000)
            $display("[GPIO] led=%b wdata=%08x", led, dut.mem_wdata);

    initial begin
        $dumpfile("tb_soc_minimal.vcd");
        $dumpvars(1, tb_soc_minimal);  // solo señales top
        resetn = 0; repeat(10) @(posedge clk);
        resetn = 1;
        // Primera escritura GPIO ocurre en ~100 ciclos (antes del delay_cycles)
        repeat(5000) @(posedge clk);
        $finish;
    end
endmodule
```

Simulación: `make sim` (Vivado XSIM batch) **Formas de onda:** `make wave` (GTKWave)

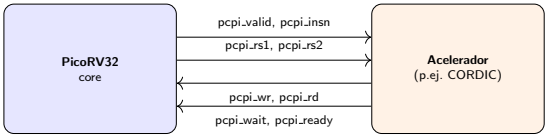
El testbench real agrega una tarea que decodifica `tx` bit a bit y cuenta los caracteres UART.

Backup Slides

Material adicional de soporte

Backup: acelerando PicoRV32 con PCPI — ejemplo CORDIC

PCPI: Pico Co-Processor Interface



- Instrucción custom (opcode no soportado) → CPU expone `pcpi_insn/rs1/rs2`
- El acelerador decodifica, calcula, y devuelve `pcpi_rd` + `pcpi_ready` (o `pcpi_wait` si tarda varios ciclos)
- Si nadie responde en 16 ciclos → excepción de instrucción ilegal
- Mismo espíritu que los `sel_*` del lab, pero acoplado al decode del CPU, no al bus de memoria

CORDIC: trigonometría con solo shift+add

$$x_{i+1} = x_i - d_i(y_i \gg i)$$

$$y_{i+1} = y_i + d_i(x_i \gg i)$$

$$z_{i+1} = z_i - d_i \arctan(2^{-i}), \quad d_i = \text{sign}(z_i)$$

- Sin multiplicador: calcula seno/coseno rotando un vector en micro-pasos, iteración a iteración
- Uso típico en SDR: demodulación, PLLs, beamforming

Resultado real (mismo PicoRV32 + Arty A7 de este curso)

CORDIC vía PCPI: **6.1**× más rápido y **78 %** menos energía por operación que hacerlo en firmware puro sobre el mismo core.

Rosso, Zerbini, Riva — *Exploring hardware/software trade-offs for CORDIC acceleration in a customized processor*, CAE 2025.

Comparativa: PicoRV32 vs otros cores RISC-V I

Cores RISC-V open source — síntesis en Artix-7

Core	LUTs	FFs	BRAM	Fmax	CPI	Ext.	Pipe
PicoRV32	∼2000	∼750	0	∼130 MHz	3–5	RV32IMC	No (FSM)
VexRiscv	∼5000	∼2000	0	∼200 MHz	∼1.5	RV32IMAC	5 etapas
SERV	∼200	∼100	0	∼200 MHz	∼32	RV32I	Serie 1b
CV32E40P	∼20000	∼8000	4	∼100 MHz	∼1.0	RV32IMFC	4 etapas

Licencias:

- PicoRV32: ISC License (permisiva, compatible con comercial)
- VexRiscv: MIT License
- SERV: ISC License
- CV32E40P: Solderpad Hardware License (Apache-compatible)

Comparativa: PicoRV32 vs otros cores RISC-V II

¿Cuándo elegir cada uno?

PicoRV32 — nuestro caso

- El área importa más que el rendimiento
- Queremos instanciar **múltiples cores** en paralelo (2 cores en un Artix-7 35T)
- Comportamiento predecible: sin caché, sin especulación, sin hazards
- Fácil de auditar: 1 archivo Verilog, $\sim$3000 líneas

VexRiscv — mayor throughput

- Necesitamos $>$100 MIPS: pipeline de 5 etapas con forwarding
- Compatible con Linux (MMU opcional), FreeRTOS
- Generado con SpinalHDL: configurable pero menos legible

Comparativa: PicoRV32 vs otros cores RISC-V III

SERV — mínima área absoluta

- Procesador **serie de 1 bit**: el más pequeño del mundo ($\sim$200 LUTs)
- Útil cuando el área es crítica pero el rendimiento no importa
- CPI $\approx$ 32 (ejecuta bits de a 1 por ciclo)

CV32E40P — grado industrial

- Core de OpenHW Group, verificado formalmente
- 4 etapas de pipeline, extensiones DSP (PULP), FPU opcional
- Para SoCs industriales que requieren certificación funcional